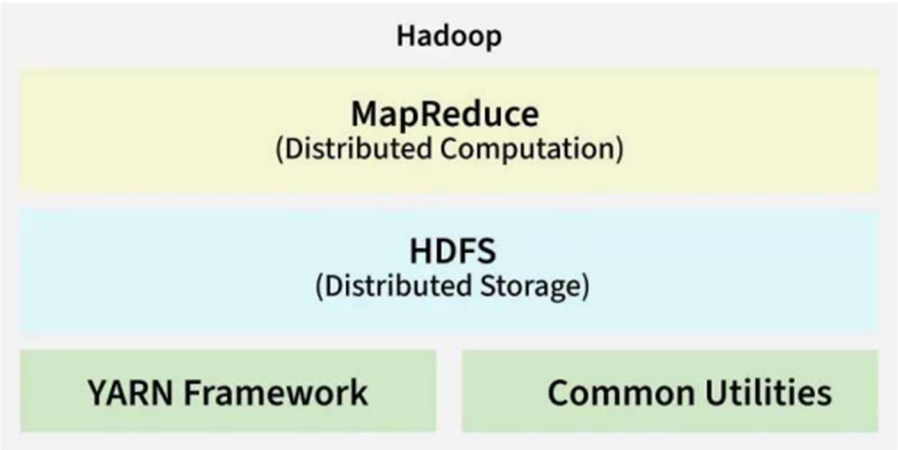
DA unit-3 by HK the OG GOAT

Topics to cover:

- Hadoop Overview
- HDFS (Hadoop Distributed File System)
- MapReduce
- NoSQL: CAP Theorem
- Hbase
- MongoDB – CouchDB
- Hive – Architecture – Hive Query Language
- Sqoop
- Apache Drill

Hadoop Overview:

Architecture:



What is Hadoop?

- **Definition:** Hadoop is an **open-source software framework** used for storing and processing massive amounts of data in a distributed computing environment.
- **The Big Idea:** Instead of using one super-expensive computer, Hadoop connects many inexpensive computers (called **nodes**) into a **cluster** to work together.
- **Key Philosophy:** It is built to be **scalable, fault-tolerant, and cost-effective**.

The 4 Core Pillars (Architecture)

The Hadoop framework is made of four main modules:

1. **HDFS (Hadoop Distributed File System):** The **Storage Layer** that breaks big files into blocks and spreads them across the cluster.
2. **MapReduce:** The **Computation Layer** or engine that processes data in a distributed and parallel manner.
3. **YARN (Yet Another Resource Negotiator):** The **Resource Management Layer** that schedules jobs and manages cluster resources like CPU and memory.
4. **Hadoop Common:** The **Utilities Layer** containing Java libraries and scripts that help all components work together.

HDFS: How it Stores Data

- **Master-Slave Model:**
    - **NameNode (Master):** The brain that stores **metadata** (like file names and where blocks are located).
    - **DataNode (Slave):** The soldiers that store the **actual data blocks** and perform read/write tasks.
- **Data Blocks:** Large files are broken into chunks, usually **128 MB or 256 MB**.
- **Replication:** Each block is usually copied **3 times** across different nodes. This ensures **Fault Tolerance**—if one node fails, your data is still safe.
- **Data Locality:** A smart feature where computation is moved close to where the data is actually stored to save time and network speed.

MapReduce: How it Processes Data

- **Map Phase:** Divides input data into chunks and processes them to produce **intermediate key-value pairs**.
- **Shuffle & Sort:** Reorganizes the data so all identical keys are grouped together.
- **Reduce Phase:** Aggregates and merges those grouped results into a **final output**.
- **Job Tracker & Task Tracker:** The Master (Job Tracker) coordinates the work while Slaves (Task Trackers) execute the tasks.

YARN: The Manager

- **Resource Manager:** The master node that manages global resource allocation.
- **NodeManager:** The slave node that monitors resources on individual machines.
- **ApplicationMaster:** Manages the lifecycle of each specific job.

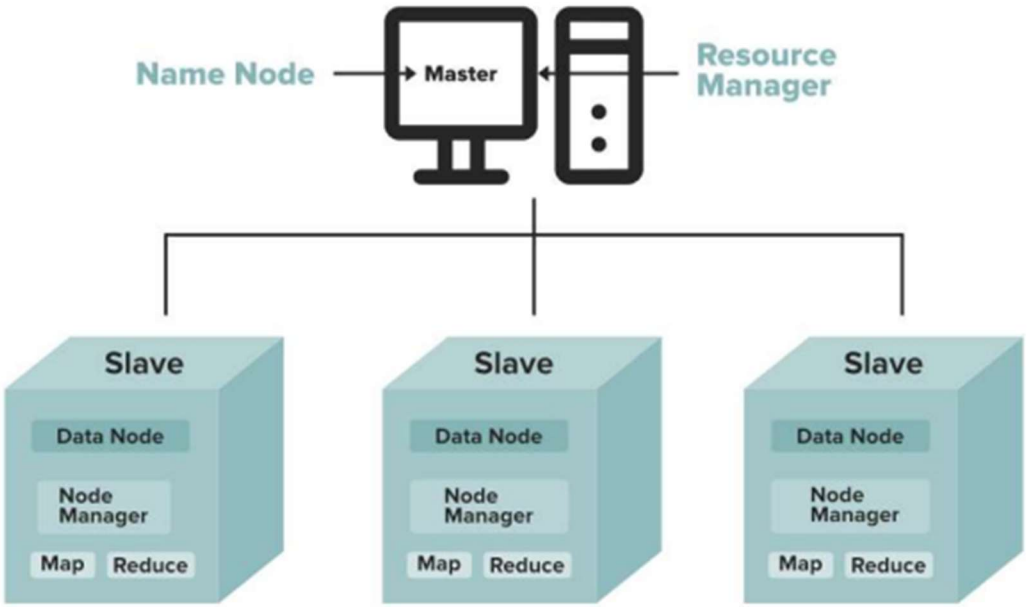
How Does Hadoop Work? (The 6-Step Flow)

1. **Data Loading:** Files are split into blocks and stored in **HDFS**.
2. **Job Submission:** A MapReduce job is sent to the **ResourceManager**.
3. **Task Division:** The job is split into **map tasks** based on data blocks.
4. **Mapping:** Map tasks produce intermediate results, which are then **shuffled and sorted**.
5. **Reducing:** Reduce tasks aggregate the data to create the final result.
6. **Final Storage:** Results are stored back in **HDFS**.

Pros, Cons, and Uses

- **Advantages:** It is highly **scalable**, uses low-cost **commodity hardware**, is **fault-tolerant**, and handles structured or unstructured data.
- **Disadvantages:** It is better for **batch processing** than real-time tasks, can be complex to program, and has **high latency** for some queries.
- **Applications:** Used in **Banking** (fraud detection), **Retail** (inventory management), **Healthcare**, and **Social Media** (trend analysis).

HDFS Architecture:



What is HDFS?

- **The Storage Layer:** HDFS is the primary storage system of Hadoop that breaks big files into smaller blocks and spreads them across a cluster.
- **Why we need it:** Storing a 30TB file on one machine is impractical due to limited disk capacity and slow processing. HDFS splits that data (e.g., a 40TB file into four 10TB chunks) so multiple nodes can work on it simultaneously, reducing processing time significantly.
- **Commodity Hardware:** It is built to run on low-cost, standard hardware because it expects and handles hardware failures automatically.

The Architecture (Master-Slave Model)

HDFS operates using two main components:

- **NameNode (Master):** The "brain" of the cluster. It stores **Metadata**, which includes file names, sizes, block IDs, and their specific locations on DataNodes. It also monitors the health of the cluster via "heartbeat" signals from slaves.

- **DataNode (Slave):** These are the workers that store the **actual data blocks**. They perform read/write tasks and create, delete, or replicate blocks based on instructions from the NameNode.

Key Concepts & Keywords

- **Data Blocks:** HDFS splits large files into chunks, usually **128 MB or 256 MB**.
- **Replication:** Each block is typically copied **3 times** across different nodes to ensure **Fault Tolerance**.
- **Data Locality:** HDFS moves the computation closer to where the data is stored to improve speed and reduce network congestion.
- **Write-once, Read-many:** Once a file is written and closed, it cannot be modified (only appended), which ensures data consistency.

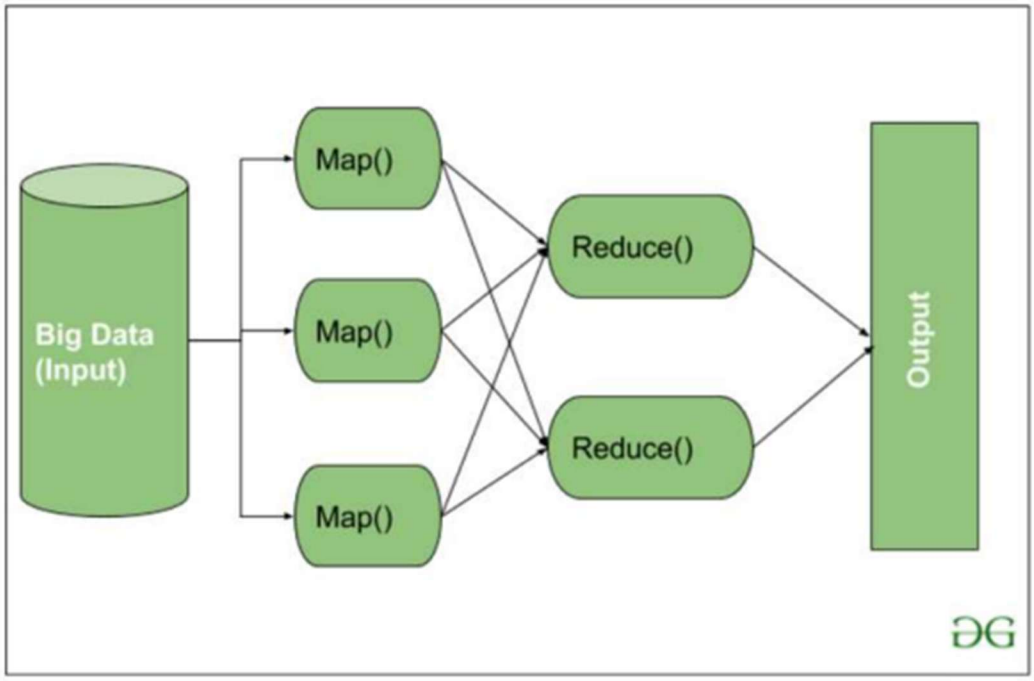
The Storage Process (Step-by-Step)

1. **File Splitting:** When you load a file, HDFS breaks it into smaller **data blocks**.
2. **NameNode Mapping:** The **NameNode** determines which **DataNodes** will store these blocks and records this in its **Metadata**.
3. **Data Distribution:** The blocks are sent to the assigned DataNodes across the cluster.
4. **Replication:** HDFS automatically creates **replicas** of these blocks on other nodes to prevent data loss if a node fails.
5. **Health Check:** DataNodes send regular **heartbeat signals** and block reports to the NameNode to confirm they are functioning correctly.
6. **Retrieval:** When you need the data, the NameNode uses its metadata to tell the client which DataNodes to contact for the blocks.

Top Features

- **Scalability:** You can easily add more nodes as your data grows.
- **High Reliability:** Built-in replication ensures your data is safe even if a machine crashes.
- **Security:** Includes authentication, authorization, and encryption for secure access.
- **High Throughput:** Designed for very fast data access and transfer.

MapReduce:



MapReduce is the processing engine and computation layer of the Hadoop framework. It is designed to handle and analyze massive amounts of data in a distributed and parallel manner across a cluster of computers.

MapReduce Architecture

The architecture serves as the backbone for Hadoop's processing, ensuring that large jobs are split, executed in parallel, and merged efficiently.

- **Master-Slave Model**: It follows a master-slave structure where one master node coordinates the work and multiple slave nodes execute the tasks.
- **Job Distribution**: The framework splits a single large job into many smaller tasks to be run across different nodes.
- **Parallelism**: By running tasks simultaneously on hundreds or thousands of nodes, it achieves high speed.
- **Data Locality**: The system moves the computation to the node where the data is physically stored to minimize network traffic.

Core Components of MapReduce Architecture

- **Client**: This is the starting point where the user submits a job. The client provides the Mapper and Reducer logic in a JAR file and specifies where the data is located.
- **Job**: This represents the entire processing request submitted by the client. It is internally broken down into smaller units called job parts or tasks.
- **Hadoop MapReduce Master (Job Tracker)**: This master node manages the entire cluster's processing.
    - It accepts jobs from the client.
    - It divides the job into smaller tasks.
    - It assigns these tasks to available nodes in the cluster.
    - It monitors progress and reassigns any tasks that fail.
- **Job Parts (Tasks)**: These are the individual units of work divided into Map tasks and Reduce tasks.

Map Task Components

The Map phase is the first stage of processing where data is transformed into intermediate results.

- **Input Splitting**: The input file is divided into chunks called splits, usually 128 MB or 256 MB in size.
- **RecordReader**: It reads the raw data from the input split and converts it into (key, value) pairs that the Mapper can understand.
- **Mapper**: This is the core logic written by the developer. It processes each (key, value) pair and generates new intermediate (key, value) pairs.
- **Combiner (Optional)**: Often called a "mini-reducer," it runs on the local node to group Mapper outputs. This reduces the amount of data that needs to be sent over the network during shuffling.
- **Partitioner**: This component decides which Reducer will receive a specific intermediate key-value pair. It typically uses a hash of the key to ensure data is distributed evenly.

Reduce Task Components

The Reduce phase takes the intermediate data and summarizes it into the final output.

- **Shuffle and Sort**: This stage transfers data from the Mappers to the Reducers. It groups all values associated with the same key together and sorts them so they can be processed sequentially.
- **Reducer**: It takes the grouped (key, list of values) and performs an aggregation or filtering operation (like summing numbers) to produce the final result.
- **OutputFormat**: This is the final step where a RecordWriter saves the consolidated results back into HDFS as a key-value line.

Step-by-Step Execution Flow

1. **Job Submission**: The client submits the JAR file and configuration to the Master.
2. **Task Assignment**: The Job Tracker splits the job into tasks and assigns them to Task Trackers based on where the data blocks are located.
3. **Map Phase**: Each Mapper processes its assigned data split in parallel and creates intermediate key-value pairs.
4. **Intermediate Stage**: The system shuffles and sorts the intermediate data to organize it by key.
5. **Reduce Phase**: Reducers aggregate the grouped data to create the final summarized output.
6. **Storage**: The final results are written back into the Hadoop Distributed File System (HDFS).

Key Optimizations

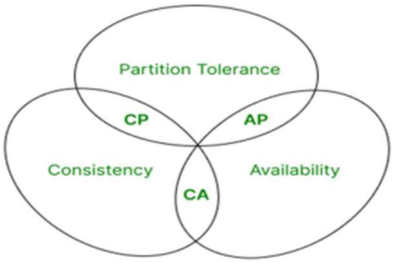
- **Speculative Execution**: If a node is running a task too slowly, Hadoop launches a duplicate of that task on another node and uses the result from whichever one finishes first.
- **Fault Tolerance**: If a node or task fails, the system automatically reassigns that work to another node that has a replica of the data.
- **Load Balancing**: The master node spreads tasks across all available workers to ensure the cluster is used efficiently.

Input File Formats

Hadoop supports various ways to read data depending on the file type:

- **TextInputFormat**: The default format; it treats each line as a record where the key is the byte offset and the value is the text of the line.
- **KeyValueTextInputFormat**: This splits each line into a key and a value based on a specific separator.
- **SequenceFileInputFormat**: A binary format used for efficient data sharing between different MapReduce jobs.
- **SequenceFileAsTextInputFormat**: This reads binary files but presents them to the Mapper as text.

NoSQL: CAP theorem:



NoSQL, which stands for "Not Only SQL," refers to non-relational database systems built to handle massive volumes of unstructured or semi-structured data through horizontal scalability and flexible data models. The CAP theorem is the fundamental principle used to understand the limitations of these distributed systems.

CAP Theorem Properties

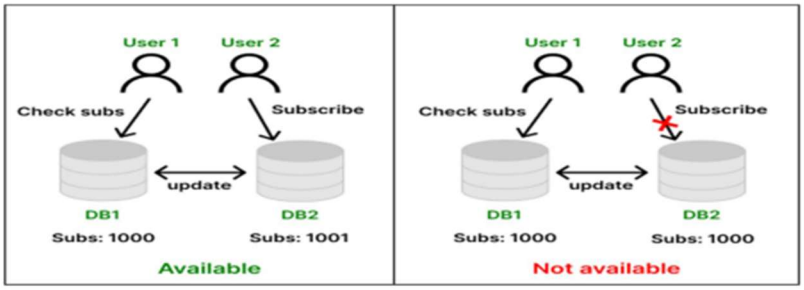
- **Consistency (C)**:

Consistency problem

  - Guarantees that every node in a distributed cluster returns the same, most recent, and successful write.
  - All clients see the same view of the data at the same time across all nodes.

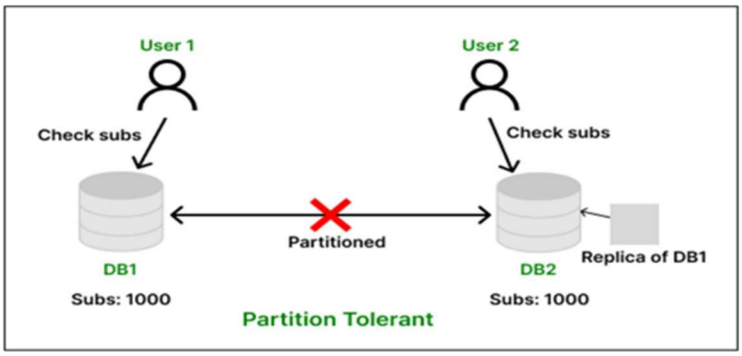
- Example: If a user spends money from their bank account, the balance update must be communicated to all database copies immediately so no other node shows the old balance.

- **Availability (A)**:



  - Ensures that every request (read or write) receives a response, even if it's a message stating the operation failed.
  - Every non-failing node must respond within a reasonable amount of time.
  - Example: In a social media network, a user should always be able to hit "subscribe" and get a response, even if they are connected to a node far away from the content creator.

- **Partition Tolerance (P)**:



  - The system continues to operate even if the network connecting the nodes fails, resulting in "partitions" where groups of nodes cannot talk to each other.

    - The system must be able to gracefully recover once the network fault is fixed.
    - Example: If a network outage disconnects two database nodes, the system should still show a user their subscriber count using a local replica of the data.

The Trade-Offs in the CAP Theorem

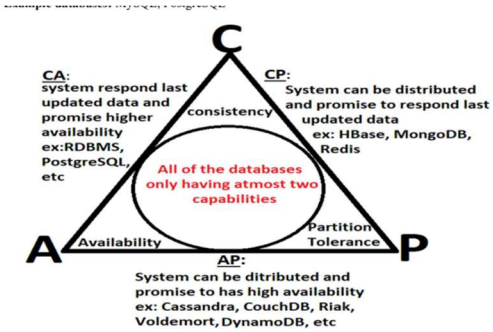
The CAP theorem states that a distributed system can only provide two out of these three properties at any given time.

- **CA (Consistency and Availability)**:
    - These systems always accept and respond to requests with consistent data across all nodes.
    - **The Problem**: They are not truly realizable in large distributed networks because network failures (partitions) are inevitable.
    - To achieve CA, a system is usually "monolithic" (single-system architecture), meaning if the system is updated, all users see it, but it lacks the benefits of distributed computing.
    - Examples: Standard RDBMS like MySQL and PostgreSQL.
- **AP (Availability and Partition Tolerance)**:
    - The system remains operational during a network partition and does not drop user requests.
    - **The Trade-off**: It prioritizes availability over consistency, meaning it might respond with "stale" (old) data that hasn't been synchronized yet.
    - Commonly used for social media (Facebook, Instagram, YouTube) where staying online is more important than showing the absolute latest data.
    - Examples: Cassandra, DynamoDB, CouchDB, and Riak.
- **CP (Consistency and Partition Tolerance)**:
    - The system ensures that all data is consistent across the network even during a partition.

- **The Trade-off**: If a partition occurs and the system cannot guarantee the latest data, it will drop the request (become unavailable) rather than show inconsistent information.
- Used for critical applications like banking, stock markets, or ticket booking where old data could lead to errors like double-booking a seat.
- Examples: HBase, MongoDB, and Redis.

CAP Trade-off Scenarios

Type	Guarantees	Compromises	Example Databases
CP System	Consistency + Partition Tolerance	May not always be available during faults.	MongoDB, HBase, Redis.
AP System	Availability + Partition Tolerance	May return stale/old data (no consistency).	Cassandra, DynamoDB, CouchDB.
CA System	Consistency + Availability	Not fault-tolerant (no partition tolerance).	Rare in distributed practice; RDBMS like MySQL.

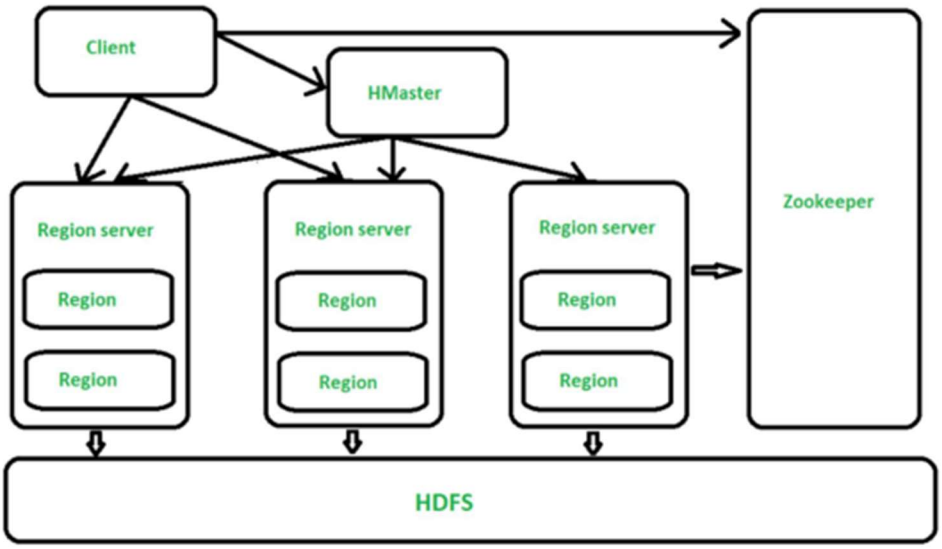


Types of NoSQL Databases

- **Document-based**:
  - Stores data as flexible, JSON-like documents (often BSON).
  - Allows all related data to be stored together in one document rather than separate tables.
  - Examples: MongoDB, CouchDB.
- **Key-Value Store**:
  - The simplest form where data is stored as unique keys paired with values.

    - They are schema-free and highly scalable.
    - Examples: Redis, DynamoDB, Riak.
- **Column-family (Column Store)**:
    - Stores data in cells grouped into columns, which are then grouped into column families.
    - Optimized for fast read/write of specific columns rather than whole rows.
    - Examples: Cassandra, HBase, Bigtable.
- **Graph-based**:
    - Focuses on the relationships between data points using graph structures (nodes and edges).
    - Examples: Neo4j, ArangoDB.

Hbase:



Architecture of HBase

- **Overview**: HBase is a distributed, scalable, NoSQL database built on top of Hadoop. It is designed for fast, random read/write access to massive amounts of structured or semi-structured data.

- **Core Structure**: The architecture relies on three primary components: HMaster, Region Server, and ZooKeeper, all of which interact with HDFS for storage.
- **Hadoop Integration**: It runs on top of HDFS DataNodes, allowing it to leverage Hadoop's distributed storage capabilities.

HMaster

- **Role**: Acts as the main coordinator or "manager" of the HBase cluster.
- **Region Management**: It is responsible for assigning data chunks, known as regions, to specific Region Servers.
- **Table Operations**: It manages administrative tasks such as creating, deleting, and modifying tables.
- **Cluster Health**: It monitors the health of Region Servers and handles failover if a server crashes.
- **Load Balancing**: It ensures that data and tasks are balanced evenly across all servers in the cluster.
- **High Availability**: Large clusters often run multiple backup HMasters to prevent a single point of failure.

Region Server

- **Role**: These are the "worker" nodes responsible for managing actual data.
- **Regions**: HBase tables are divided horizontally into regions (typically 256 MB by default), and each Region Server manages a specific row-key range.
- **Client Interaction**: They directly handle all read and write requests from clients.
- **Data Storage**: They store data in column families, which are the basic storage units in HBase.
- **Automatic Scaling**: As a table grows and a region becomes too large, new regions are automatically created and managed.

ZooKeeper

- **Role**: Acts as a "traffic controller" and synchronization service for the cluster.
- **Coordination**: It enables communication and coordination between the HMaster, Region Servers, and clients.

- **Server Discovery**: It helps clients identify which Region Server holds the specific data they need.
- **Failure Detection**: It monitors for server failures to trigger quick recovery processes.
- **Configuration Management**: It maintains the cluster's configuration settings and provides distributed synchronization.

How HBase Works (Write Path)

- **Flow**: Client → Region Server → WAL → MemStore → HFile.
- **Step 1: Write Request**: The client sends a request to the appropriate Region Server to save data.
- **Step 2: WAL (Write Ahead Log)**: The Region Server first records the data in the WAL, which acts as a safety draft to prevent data loss if the server crashes.
- **Step 3: MemStore**: The data is then placed into the MemStore, a temporary buffer in the system's RAM, making the write operation very fast.
- **Step 4: Flushing to Disk**: Once the MemStore becomes full, the data is flushed and saved permanently to HDFS as an HFile.
- **Step 5: Compaction**: In the background, HBase merges smaller HFiles into larger ones to reduce storage space and speed up future read operations.

How Data is Read in HBase (Read Path)

- **Flow**: Client → ZooKeeper → Region Server → BlockCache → MemStore → HFile.
- **Step 1: Locating Data**: The client first contacts ZooKeeper to find out which Region Server is responsible for the required data.
- **Step 2: BlockCache**: The Region Server checks the BlockCache first, which stores recently used data in memory for instant access.
- **Step 3: MemStore**: If the data is not in the cache, the server checks the MemStore for recent writes that haven't been saved to disk yet.
- **Step 4: HFile**: If the data is still not found, the server finally reads it from the actual HFiles stored on the disk in HDFS.

Key Features of HBase Architecture

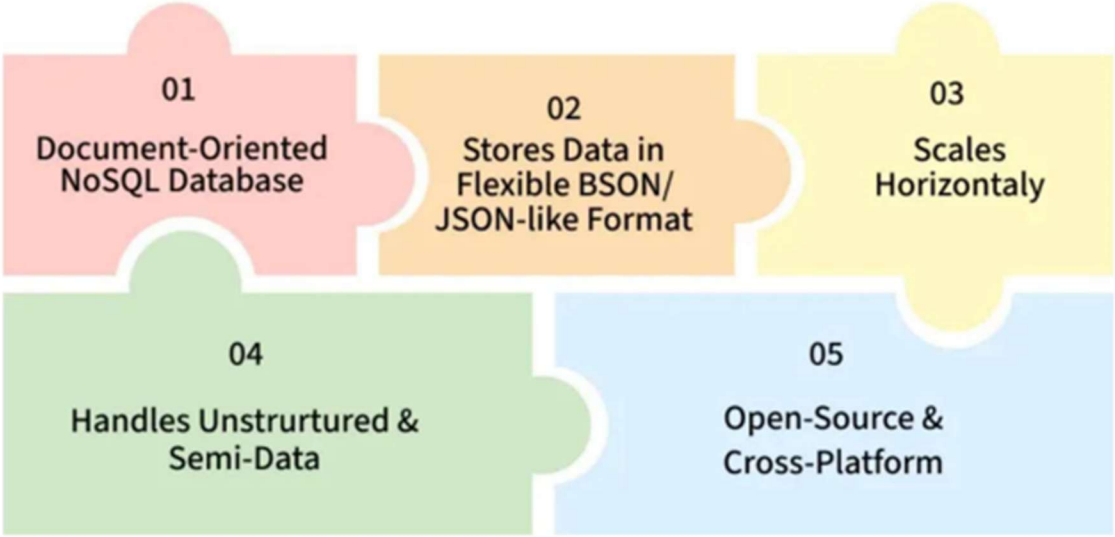
- **Distributed and Scalable**: It can grow across thousands of machines to store enormous datasets.

- **Column-Oriented**: Data is stored in column families, which optimizes performance for operations on specific columns.
- **Strong Consistency**: It ensures every read and write operation is consistent across the entire cluster.
- **Built-in Caching**: Frequently accessed data is kept in memory (BlockCache) to ensure high performance.
- **Flexible Schema**: Columns can be added dynamically without needing to redefine the entire table structure.
- **Data Compression**: It uses compression to reduce storage usage and increase retrieval speed.

Advantages and Disadvantages

- **Advantages**:
    - **Massive Scalability**: Easily handles gigabytes to petabytes of data.
    - **Real-Time Performance**: Suitable for high-speed, real-time read/write workloads.
    - **Fault Tolerance**: High availability through replication and automatic failover.
    - **Cost-Effective**: Runs on inexpensive, commodity hardware.
- **Disadvantages**:
    - **No SQL Support**: It does not support standard SQL queries or complex joins.
    - **ACID Limitations**: It does not provide full ACID transactions across the entire database.
    - **Sorting Constraints**: Rows are only sorted by their specific row key.
    - **Management Complexity**: Requires careful memory management and skilled professionals to maintain large clusters.

MongoDB:



MongoDB is a document-oriented NoSQL database that stores data in a flexible, JSON-like format called BSON, allowing for efficient storage and retrieval of unstructured or semi-structured data.

A Simple MongoDB Document Structure

- **Format**: Data is stored as individual documents rather than rows and columns.
- **_id**: Every document has a unique identifier.
- **Fields**: Can contain various data types, such as:
    - **Strings**: For titles, authors, or URLs.
    - **Arrays**: To store lists of items like "tags".
    - **Booleans**: To store true/false values like "published" status.
    - **Numbers**: To track counts like "views".

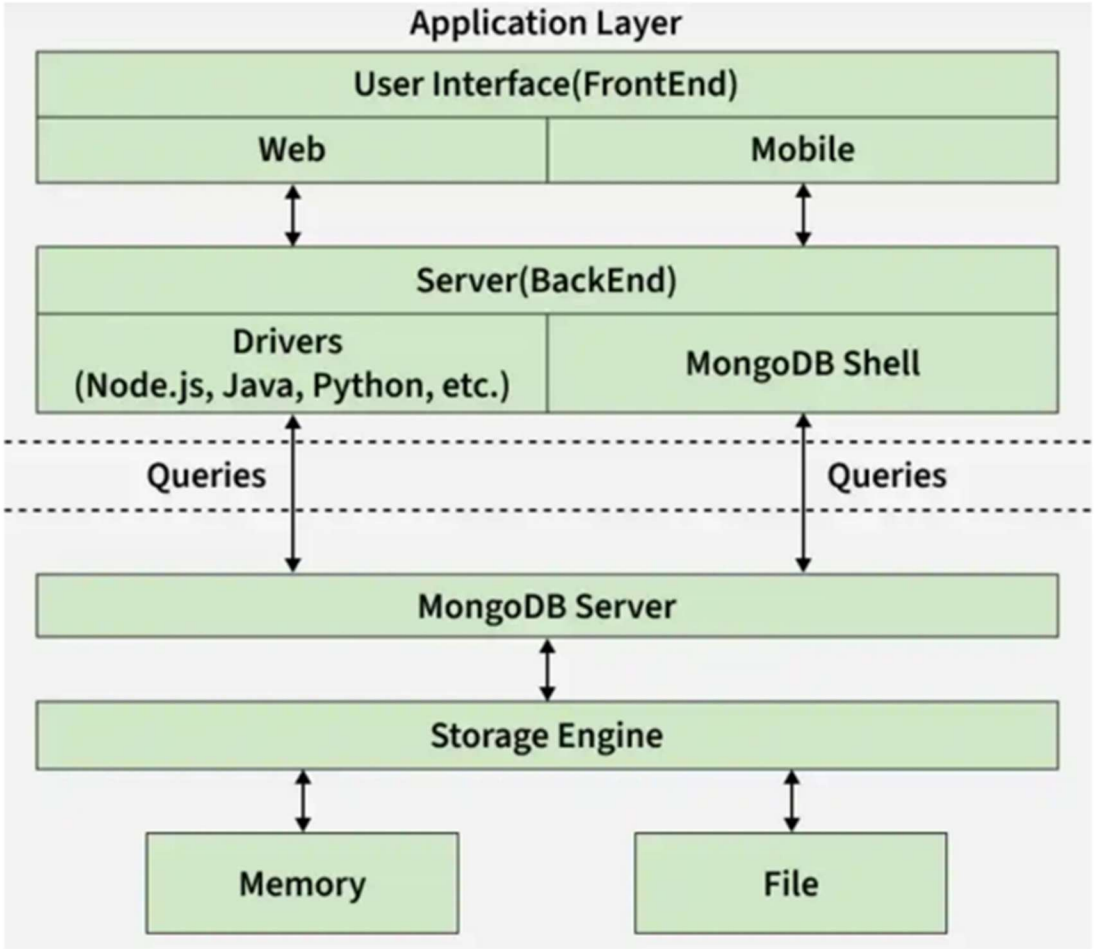
Basic SQL Operations

MongoDB provides operations to manage data that correspond to standard SQL commands:

- **Create (Insert)**: Used to add new documents to a collection.
- **Read (Find)**: Used to retrieve specific documents based on query criteria.
- **Update**: Used to modify existing data within documents.

- **Delete**: Used to remove documents from a collection.

Working of MongoDB



MongoDB operates through two primary layers to handle data requests:

- **Application Layer**: This is the interface between the user and the database.
    - **Frontend**: Includes the web or mobile user interface where users interact with the app.
    - **Backend**: The server-side logic that uses drivers (like Java or Python) or the MongoDB Shell to send queries to the database.
- **Data Layer**: This layer handles the actual data management.
    - **MongoDB Server**: It receives queries from the backend and forwards them to the storage engine.

- **Storage Engine**: Manages the physical reading and writing of data. It optimizes performance by using RAM for frequently accessed data and disk for long-term storage.

MongoDB Data Flow

The internal process for a single operation follows these steps:

1. **User Interaction**: A user interacts with the frontend to send a request.
2. **Backend Processing**: The backend server receives the request and communicates with the database using drivers.
3. **Server Communication**: The MongoDB Server receives the query and passes it to the storage engine.
4. **Data Execution**: The storage engine reads or writes the data from memory or disk and returns the result.

Features of the MongoDB Database

- **Document Oriented**: Stores all related information together in one flexible document, avoiding the need for complex table joins.
- **Indexing**: Creates "indexes" to find data quickly without scanning every document in a collection.
- **Scalability**: Uses "sharding" to distribute data across many servers, allowing the system to grow as data increases.
- **Replication and High Availability**: Keeps multiple copies of data on different servers to ensure the system stays online even if one server fails.
- **Aggregation**: Provides tools to process and summarize data, similar to SQL "GROUP BY" functions (e.g., sum, average, min, max).

Real-World Applications

MongoDB is used to power a wide range of modern services:

- **Websites & Apps**: Managing product catalogs and user profiles for e-commerce platforms.
- **Real-Time Analytics**: Analyzing sales trends or user activity instantly.
- **Content Management**: Storing dynamic content and providing personalized recommendations (like movie suggestions).

- **Big Data & IoT**: Processing massive volumes of unstructured data from sensors, logs, and social media feeds.

Language Support by MongoDB

MongoDB provides official drivers for many popular programming languages, including:

- **C, C++, and C#**.
- **Java and Scala**.
- **Python and Ruby**.
- **Node.js, PHP, and Perl**.
- **Go, Rust, and Erlang**.

CouchDB:

Its an open source NoSQL database developed my Apache software foundations

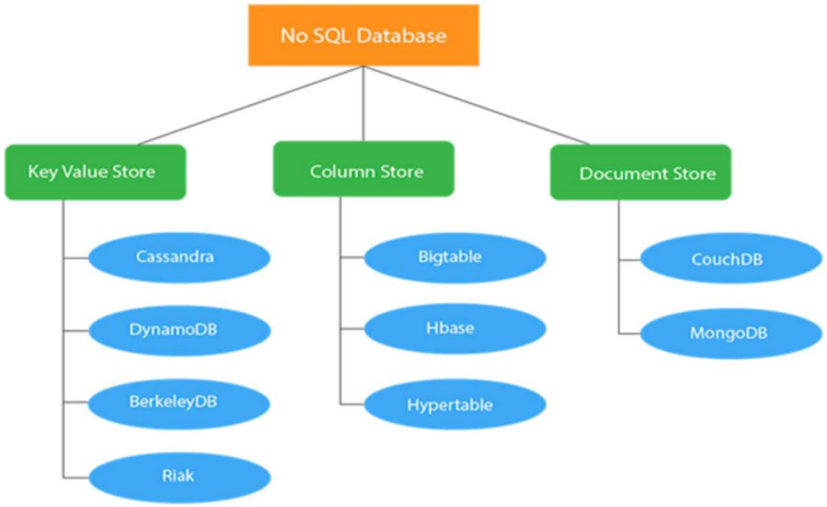
RDBMS: Relational Database Management System

- **Definition**: A system based on the relational model introduced by E.F. Codd, serving as the basis for SQL.
- **Data Storage**: Data is organized into database objects called tables, which consist of related entries in columns and rows.
- **Data Type**: It is designed to store only structured data.
- **Examples**: Common systems include Oracle, MySQL, MS SQL Server, IBM DB2, and Microsoft Access.

NoSQL Databases

- **Definition**: Non-relational database systems designed to handle large volumes of unstructured or semi-structured data.
- **Core Characteristics**: These databases are schema-free, support easy replication, have simple APIs, and can handle massive amounts of data.
- **Main Objectives**: They focus on simplicity of design, horizontal scaling, and finer control over availability.

- **Performance**: They are typically faster than RDBMS because they use different data structures and can store various formats like audio, video, and documents.
- **Main Types**:
    - **Key-value Store**: Stores data in simple pairs.
    - **Column Store**: Groups data into cells and columns.
    - **Document Store**: Stores data as self-contained documents.



Key-value Store

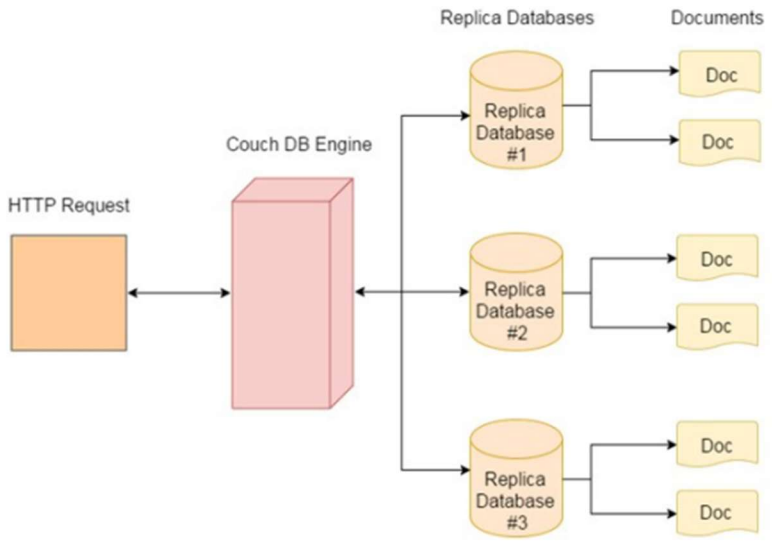
- **Structure**: Data is stored in simple key-value pairs where each value has an indexed key.
- **Schema**: These databases do not have a fixed schema.
- **Examples**: Popular versions include Cassandra, DynamoDB, BerkeleyDB, and Riak.

CouchDB Data Model

- **Hierarchy**: The "Database" is the outermost container, holding a collection of independent documents.
- **Document Autonomy**: Each document maintains its own data and a self-contained schema.
- **Metadata**: Documents contain revision information, which allows the system to merge differences that occurred while databases were disconnected.

- **Concurrency**: It implements Multi-Version Concurrency Control (MVCC) to avoid the need for locking database fields during write operations.

CouchDB Architecture



- **Client / HTTP Request**: Users interact with the database through an HTTP/REST API using standard commands: **GET** (retrieve), **POST/PUT** (store), and **DELETE** (remove).
- **CouchDB Engine**: The core component that processes client requests, manages databases, handles replication, and ensures data consistency.
- **Replica Databases**: These are copies of the same database stored across different nodes to ensure:
    - **High Availability**: If one node fails, others continue working.
    - **Fault Tolerance**: Multiple copies prevent data loss.
    - **Load Balancing**: Replicas handle many user requests efficiently.
- **Documents**: Data is stored inside these replicas as JSON documents.
- **Data Flow**: A client sends a request → the Engine processes it → the Engine reads/writes to a Replica Database → changes are synchronized across all replicas.

Key Features Shown in the Architecture

- **RESTful API**: Communication is handled entirely through HTTP/REST.
- **Document Storage**: Uses JSON for data storage.
- **Replication**: Uses multiple database replicas to maintain copies of data.

- **Scalability**: New replicas can be added easily to support more data and users.

Features of CouchDB

- **Document Storage**: Primary unit of data where fields contain various types (text, number, Boolean, lists) with no set limit on text size.
- **Browser-Based GUI**: Provides an interface called "Futon" to handle data, permissions, and configuration via a browser.
- **Simple Replication**: Known for having the simplest form of replication among databases.
- **ACID Properties**: Updates follow Atomicity (saved completely or not at all), and all entries are serialized so they cannot be overwritten once on the disc.
- **JSONP Support**: Allows the database to be accessible cross-domain for GET requests if configured.
- **Authentication and Sessions**: Supports session cookies for authentication, similar to web applications.
- **Security**: Offers database-level security with separate permissions for "readers" and "admins".
- **Validation**: Inserted data can be validated in combination with authentication to ensure only authorized creators modify documents.
- **Map/Reduce**: Features a powerful Map/Reduce system for data processing and views.

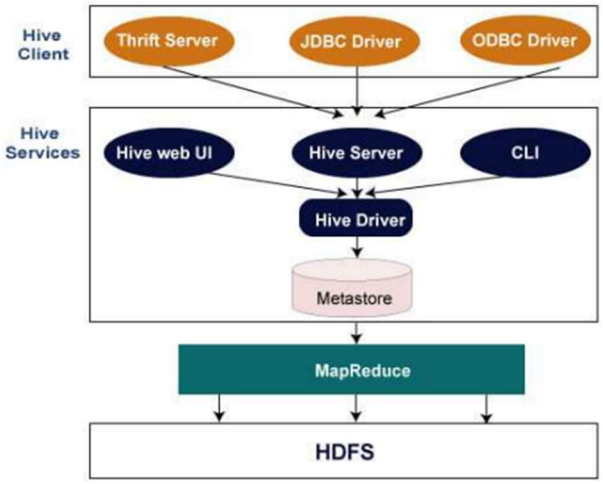
Hive:

What is Hive?

- **Definition**: Apache Hive is an open-source data warehouse software used for querying and managing massive datasets stored in the Hadoop Distributed File System (HDFS).
- **Language**: It uses a declarative language called Hive Query Language (HQL), which is very similar to standard SQL.
- **Core Function**: It converts HQL queries into MapReduce, Tez, or Spark jobs that run on a Hadoop cluster.

- **Purpose**: It was created to replace complex Java MapReduce programming with a simpler interface for structured and semi-structured data analysis.
- **Use Cases**: It is primarily used for large-scale data analysis, ad-hoc querying, and data encapsulation.

Hive Architecture and Query Flow



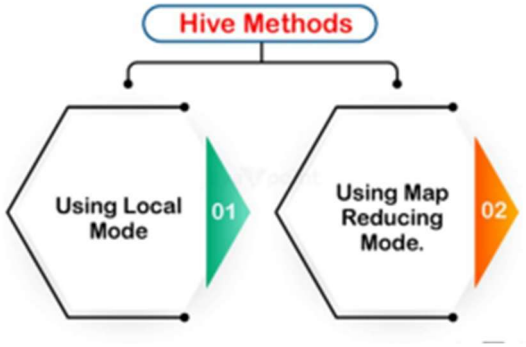
The architecture consists of several layers that work together to process a user's query:

- **Hive Client**: Allows applications written in Java, Python, or C++ to connect using a Thrift Server, JDBC Driver, or ODBC Driver.
- **Hive Services**:
    - **CLI & Web UI**: Interfaces where users execute commands and queries.
    - **Hive MetaStore**: A central database that stores all structural information (metadata), such as table schemas, column types, and where data is located in HDFS.
    - **Hive Server (Thrift Server)**: Accepts requests from different clients and forwards them to the Driver.
- **Hive Driver**: Receives the query from the interface and sends it to the Compiler.
- **Hive Compiler**: Parses the query, checks the syntax, and performs semantic analysis to convert HQL into a series of MapReduce tasks.
- **Hive Execution Engine**: Coordinates the execution of tasks in the order of their dependencies and collects the final results.

Features of Hive

- **Cost-Effective**: It is a free, open-source tool.
- **Scalability**: Efficiently handles petabytes of data across a Hadoop cluster.
- **Concurrency**: Multiple users can query the data at the same time.
- **Performance Optimization**: Uses partitioning and bucketing at the table level to speed up queries.
- **Format Support**: Supports various file formats including textFile, ORC, Avro, Parquet, and sequence files.
- **Ease of Use**: Users only need basic SQL knowledge; no complex programming is required.
- **Extensibility**: Supports User-Defined Functions (UDFs) for custom data processing and cleaning.
- **External Tables**: Allows processing of data stored outside of HDFS without actually moving it into the system.
- **Standard Structure**: Hive table structures are identical to those in traditional relational databases.
- **ETL Support**: Functions as a fast and efficient tool for Extract, Transform, and Load (ETL) operations.
- **Analytical Power**: Supports Online Analytical Processing (OLAP) and ad-hoc queries.
- **Tez Integration**: Can be combined with Apache Tez for real-time processing and data visualization.

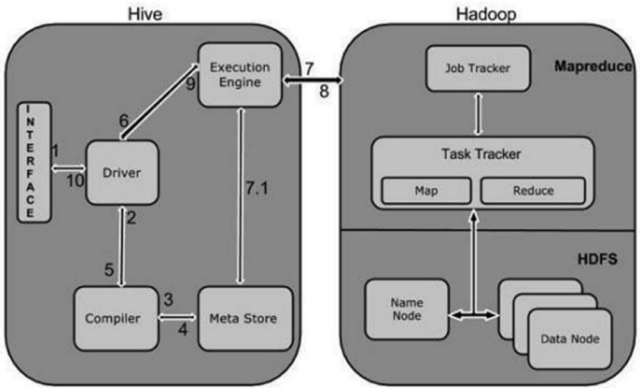
Different Hive Methods



Depending on the size of the data and the cluster configuration, Hive can operate in two ways:

- **Local Mode**:
  - Used when Hadoop is running on a single data node (pseudo-distributed mode).
  - Ideal for processing smaller datasets that fit on one machine.
  - Offers faster processing speeds for small tasks because it avoids network overhead.
- **Map Reduce Mode**:
  - Used in a full Hadoop cluster with multiple data nodes.
  - Designed to manage and process massive volumes of data distributed across many servers.
  - Enables parallel processing to handle multiple large-scale queries simultaneously.

Working of Hive (Step-by-Step)



1. **Execute Query**: The user submits a query through the CLI or Web UI to the Driver.
2. **Get Plan**: The Driver sends the query to the Compiler to check syntax and create a plan.
3. **Get Metadata**: The Compiler requests necessary table and schema information from the Metastore.

4. **Send Metadata**: The Metastore sends the requested metadata back to the Compiler.
5. **Send Plan**: The Compiler prepares the final execution plan and sends it back to the Driver.
6. **Execute Plan**: The Driver sends this execution plan to the Execution Engine.
7. **Execute Job**: The Execution Engine converts the tasks into MapReduce jobs and submits them to the Hadoop framework (JobTracker/TaskTracker).
8. **Fetch Result**: Once the jobs finish, the Engine collects the data from the DataNodes.
9. **Send Results**: The Engine sends the collected results back to the Driver.
10. **Return Results**: The Driver delivers the final output back to the user interface for viewing.

Hive Query Language (HQL)

HQL is the primary tool for data analysis in the Hive ecosystem:

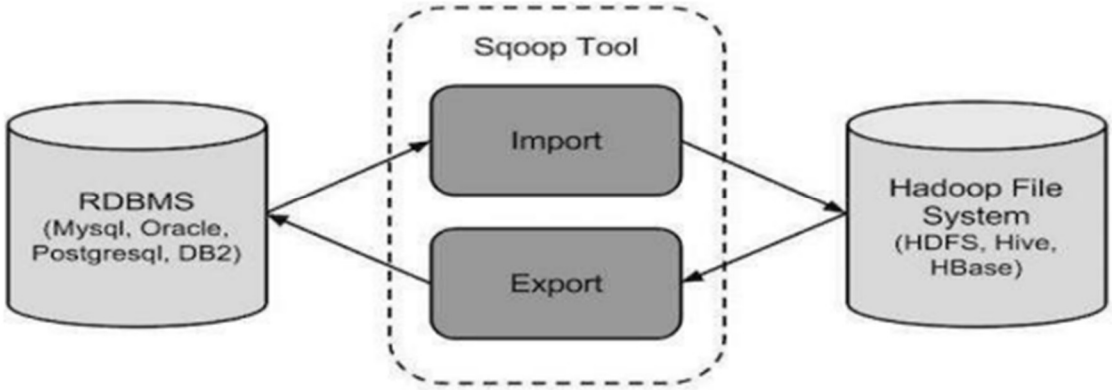
- **SQL-like Syntax**: Highly similar to standard SQL, making it easy for database professionals to use.
- **Schema on Read**: Hive does not check data against a schema when it is stored; instead, it applies the schema only when the data is read.
- **Query Types**:
    - **Data Definition Language (DDL)**: Used to create, drop, and alter databases and tables.
    - **Data Manipulation Language (DML)**: Used to load data into tables or insert new records.
    - **Data Query Language (DQL)**: Used to retrieve and filter data using SELECT statements.
    - **Data Control Language (DCL)**: Used to manage security by granting or revoking user permissions.

Advantages of Hive Query Language

- **Simple Learning Curve**: Extremely easy to learn for anyone familiar with SQL.

- **Efficiency**: Handles large-scale data processing much faster than manual coding.
- **Reduced Complexity**: Eliminates the need to write hundreds of lines of complex Java MapReduce code.
- **Analytics Focused**: Perfectly suited for data warehousing and complex analytical queries.
- **Seamless Integration**: Works perfectly with other components of the Hadoop ecosystem like HDFS and HBase.

Sqoop:



Sqoop is a command-line interface application designed to transfer bulk data between relational databases and the Hadoop ecosystem.

What is Sqoop?

- **Definition**: The name stands for "SQL to Hadoop and Hadoop to SQL".
- **Function**: It imports data from relational databases (RDBMS) like MySQL or Oracle into HDFS, Hive, or HBase, and exports data back from Hadoop to an RDBMS.
- **Purpose**: It acts as a bridge between traditional relational database servers and Hadoop's big data storage and analysis tools.

How Sqoop Works

- **Command Processing**: It primarily uses a command-line interface but can also interact through Java APIs.
- **MapReduce Mechanism**: When a user submits a command, Sqoop parses the arguments and initiates a Map-only job.

- **Parallel Processing**: It releases multiple mappers (the number is defined by the user) to fetch or send data in parallel, which increases efficiency.
- **Data Distribution**: For imports, each mapper is assigned a specific part of the data based on a key defined by the user.
- **Connectivity**: Each mapper creates an individual connection to the database using a Java Database Connectivity (JDBC) model.

Sqoop Import

- **Core Task**: Transfers a table from an RDBMS to the Hadoop database server.
- **Storage**: Each record in the relational table is imported and stored as a separate record in a text file within Hadoop.
- **Incremental Import**: It supports adding only new rows to an existing database rather than re-importing the entire dataset.
- **Integration**: It can automatically create table partitions and load data directly into Hive or HBase.

Sqoop Export

- **Core Task**: Transfers files from the Hadoop distributed system back to a structured relational database (like MySQL or SQL Server).
- **Two-Step Process**:
    1. **Metadata Examination**: It checks the target database for its structure and metadata.
    2. **Data Migration**: It performs the actual transfer of data.
- **Mapping**: The Hadoop data files are mapped into Map Tasks that bring the records to the destination RDBMS.

Key Features

- **SQL Integration**: Can connect the results of specific SQL queries directly into HDFS.
- **Direct Loading**: Moves processed data directly into Hive or HBase.
- **Security**: Uses Kerberos for security operations and data authentication.
- **Compression**: Capable of compressing data during the transfer process.

- **Versatility**: Works with a wide variety of structured data stores, including Teradata, Oracle, and DB2.

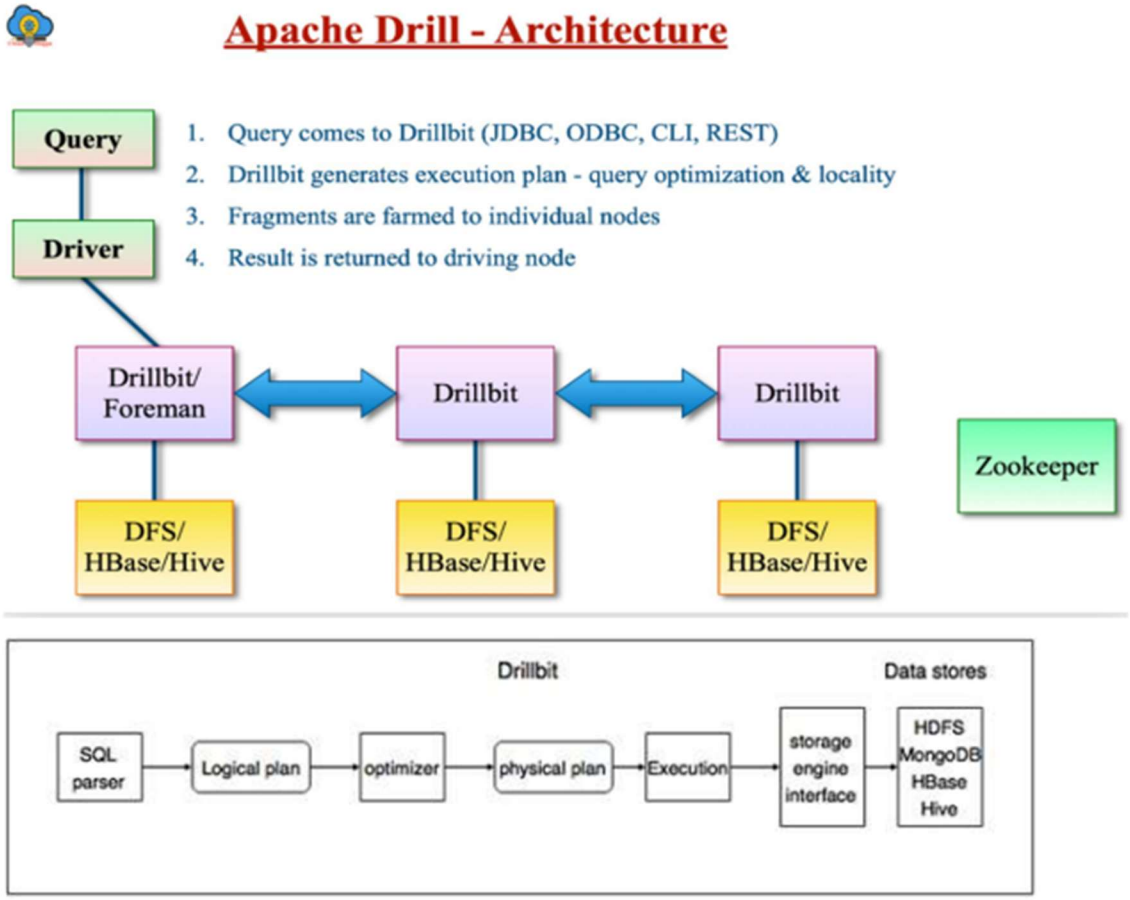
Advantages

- **Speed**: Parallel processing ensures the overall data transfer is very fast.
- **Cost-Effective**: Provides a budget-friendly way to perform ETL (Extract, Transform, Load) operations.
- **Fault Tolerance**: Because it uses MapReduce, it can automatically handle and recover from certain failures.

Disadvantages

- **Connection Inefficiency**: Relying on JDBC to establish connections is considered an inefficient method for large-scale transfers.
- **Hardware Dependence**: The performance of export operations is heavily dependent on the hardware configuration of the target RDBMS.
- **Complex Failures**: If an operation fails during implementation, it often requires a specialized solution to resolve the issue.

Apache Drill:



Apache Drill is a distributed system designed for the interactive analysis of massive datasets, allowing users to gain quick insights without needing to transform data or load it into a predefined schema.

Core Features of Apache Drill

- **Schema-Free JSON Documents**: It can ingest data in various raw formats, including JSON, Parquet, and CSV.
- **SQL Query Language**: It uses standard SQL, making it accessible to most data analysts.
- **Speed**: Its execution engines are optimized for low latency to return results rapidly.
- **Extensibility**: Developers can create custom storage plugins to expand its capabilities.

Apache Drill Architecture Components

- **Drillbit**: This is a daemon service responsible for accepting client requests, processing queries, and returning results. Unlike other systems, it has no master-slave concept; any Drillbit can handle a request.
- **SQL Parser**: It parses all incoming queries using an open-source framework called Calcite.
- **Logical Plan**: This describes the abstract flow of data for a query.
- **Optimizer**: It uses rule-based and cost-based techniques to translate a logical plan into an efficient physical plan. It is pluggable, allowing for custom optimization rules.
- **Physical Plan**: Also called the execution plan, it represents the fastest way to run a query across cluster nodes using a Directed Acyclic Graph (DAG) of physical operators.
- **Storage Engine Interface**: These are extensible plugins that allow Drill to interact with various data sources.
- **ZooKeeper**: Used by the "Foreman" (the driving Drillbit) to get a list of available nodes in the cluster for scheduling tasks.

Query Execution Flow

1. **Submission**: A client issues a query via JDBC, ODBC, CLI, or REST to any Drillbit in the cluster.
2. **Planning**: That Drillbit (now called the Foreman) parses and optimizes the query to generate a distributed execution plan.
3. **Scheduling**: The Foreman gets a list of available nodes from ZooKeeper and schedules query fragments to run on them.
4. **Execution**: Individual nodes execute their assigned fragments.
5. **Collection**: The nodes return their results to the Foreman.
6. **Final Result**: The Foreman returns the final consolidated results back to the client.

Why Use Apache Drill? (Advantages)

- **Rapid Setup**: It can be set up in minutes on Windows, Linux, or Mac by simply untarring the setup file.

- **Schema-Free JSON Model**: It automatically understands data structures without requiring a predefined schema.
- **Standard SQL Support**: It supports the SQL:2003 syntax, including JOINs, WHERE clauses, and various data types like VARCHAR and TIMESTAMP.
- **BI Tool Integration**: It provides JDBC and ODBC drivers to connect with tools like Tableau, Qlik, Excel, and MicroStrategy.
- **In-Situ Querying**: It can query complex, semi-structured data exactly where it resides without requiring prior data transformation.
- **Diverse Data Sources**: It can combine data on the fly from HDFS, Amazon S3, Hive, HBase, and more.
- **Massive Scalability**: It scales from a single computer to a cluster of 1,000 nodes.
- **Pipelined Model**: It uses an optimistic pipelined model to aggregate cluster memory; if data doesn't fit in memory, it automatically spills to disk.
- **User-Defined Functions (UDFs)**: It supports custom Java-based UDFs and can even use UDFs originally created for Hive.
- **Hive Integration**: It can run queries directly on Hive tables and join them with other sources like HBase or log files.
- **High Performance**: It uses vectorized execution, a columnar engine, and its own optimized execution engine (rather than MapReduce or Spark) to ensure maximum efficiency.